

Experiment No. 1

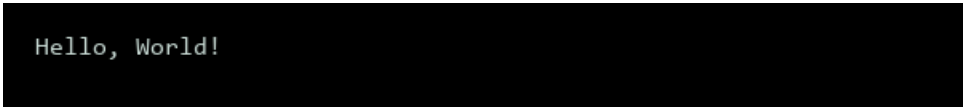
Aim: Install C compiler (GCC/Code::Blocks), set up IDE, compile and run the first "Hello, World!" program.

Concept Used: The implementation uses C to demonstrate basic programming concepts, including compilation and execution. C is a general-purpose programming language used for building operating systems, games, and other applications. This experiment targets core concepts such as variables, data types, and input/output operations.

Code:

```
#include <stdio.h>
int main() {
    printf("Hello, World!\n");
    return 0;
}
```

Output:



```
Hello, World!
```

Figure 1 - Hello World Output

Experiment No. 2

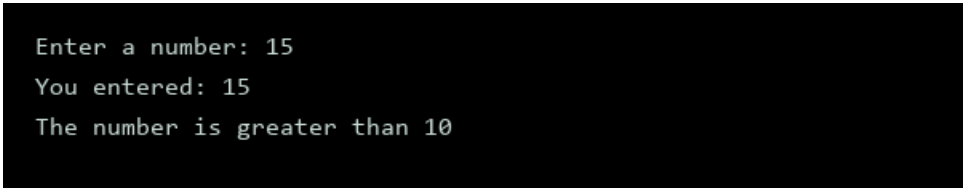
Aim: Write a Program to show the use to input (Scanf)/output (Printf) statements and block structure of C-program by highlighting the features of "stdio.h".

Concept Used: The C programming language implements standard input/output functions through the stdio.h library. This experiment demonstrates the use of input and output statements, such as scanf and printf, within a block structure. The implementation uses C to exhibit these fundamental features. Basic program control and data types are also utilized.

Code:

```
#include <stdio.h>
int main() {
    int num;
    printf("Enter a number: ");
    scanf("%d", &num);
    printf("You entered: %d\n", num);
    if (num > 10) {
        printf("The number is greater than 10\n");
    } else {
        printf("The number is less than or equal to 10\n");
    }
    return 0;
}
```

Output:

A screenshot of a terminal window with a black background and white text. It shows the output of the C program: 'Enter a number: 15', 'You entered: 15', and 'The number is greater than 10'.

```
Enter a number: 15
You entered: 15
The number is greater than 10
```

Figure 2 – Input-Output using scanf & printf

Experiment No. 3

Aim: Write a program to add two numbers and display the sum.

Concept Used: The implementation uses C to target basic arithmetic operations and input/output functions. This program demonstrates variable declaration, user input, and arithmetic addition. The C language is utilized for its simplicity and ease of use.

Code:

```
#include <stdio.h>
int main() {
    int num1, num2, sum;
    printf("Enter first number: ");
    scanf("%d", &num1);
    printf("Enter second number: ");
    scanf("%d", &num2);
    sum = num1 + num2;
    printf("The sum is: %d\n", sum);
    return 0;
}
```

Output:

A screenshot of a terminal window with a black background and white text. It shows the output of the C program: 'Enter first number: 10', 'Enter second number: 20', and 'The sum is: 30'.

```
Enter first number: 10
Enter second number: 20
The sum is: 30
```

Figure 3 - Simple Addition Output

Experiment No. 4

Aim: Write a program to calculate the area and the circumference of a circle by using radius as the input provided by the user.

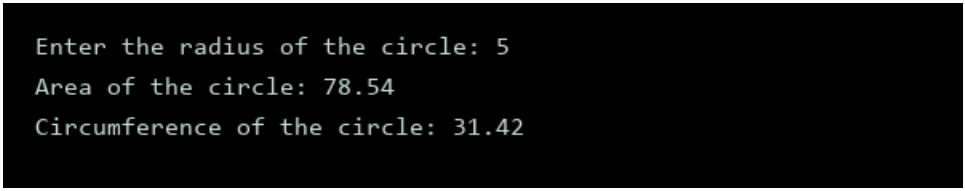
Concept Used: The program utilizes mathematical formulas for calculating the area and circumference of a circle, specifically using the radius as input. The implementation uses C, incorporating basic input/output operations and arithmetic calculations. Core concepts include variables, data types, and operators.

Code:

```
#include <stdio.h>
#include <math.h>

int main() {
    float radius, area, circumference;
    printf("Enter the radius of the circle: ");
    scanf("%f", &radius);
    area = M_PI * radius * radius;
    circumference = 2 * M_PI * radius;
    printf("Area of the circle: %.2f\n", area);
    printf("Circumference of the circle: %.2f\n", circumference);
    return 0;
}
```

Output:

A screenshot of a terminal window with a black background and light green text. It shows the output of the C program: a prompt to enter the radius, followed by the calculated area and circumference for a radius of 5.

```
Enter the radius of the circle: 5
Area of the circle: 78.54
Circumference of the circle: 31.42
```

Figure 4 - Circle Calculations

Experiment No. 5

Aim: Write a Program to perform addition, subtraction, division and multiplication of two numbers given as input by the user.

Concept Used: The program utilizes basic arithmetic operators and user input handling in C. It implements addition, subtraction, division, and multiplication operations. The implementation uses C to demonstrate these fundamental concepts. Arithmetic operations are performed based on user input.

Code:

```
#include <stdio.h>
int main() {
    float num1, num2;
    char op;
    printf("Enter first number: ");
    scanf("%f", &num1);
    printf("Enter operator (+, -, *, /): ");
    scanf(" %c", &op);
    printf("Enter second number: ");
    scanf("%f", &num2);
    if (op == '+') {
        printf("%.2f + %.2f = %.2f\n", num1, num2, num1 + num2);
    } else if (op == '-') {
        printf("%.2f - %.2f = %.2f\n", num1, num2, num1 - num2);
    } else if (op == '*') {
        printf("%.2f * %.2f = %.2f\n", num1, num2, num1 * num2);
    } else if (op == '/') {
        if (num2 != 0) {
            printf("%.2f / %.2f = %.2f\n", num1, num2, num1 / num2);
        } else {
            printf("Error: Division by zero is not allowed\n");
        }
    } else {
        printf("Invalid operator\n");
    }
}
```

```
    return 0;  
}
```

Output:

```
Enter first number: 10  
Enter operator (+, -, *, /): +  
Enter second number: 5  
10.00 + 5.00 = 15.00
```

Figure 5 - Arithmetic Operation

Experiment No. 6

Aim: Write a program to evaluate each of the following equations.

(i) $V = u + at$

(ii) $S = ut + \frac{1}{2}at^2$

(iii) $T = 2\sqrt{a + b} + 9c$

(iv) $H = \sqrt{b^2 + p^2}$

Concept Used: Equation evaluation is performed using basic algebraic operations in C, covering equations of motion and quadratic formulas. The implementation uses C to compute the results of given mathematical equations. It demonstrates variable assignments, arithmetic operations, and function calls.

Code:

```
#include <stdio.h>
#include <math.h>

int main() {
    float u, a, t, V, S, b, c, T, p, H;

    // equation (i)
    u = 10.0;
    a = 2.0;
    t = 3.0;
    V = u + a * t;
    printf("V = u + at: %.2f = %.2f + %.2f*%.2f\n", V, u, a, t);

    // equation (ii)
    u = 5.0;
    t = 4.0;
    a = 1.0;
    S = u * t + 0.5 * a * t * t;
    printf("S = ut + 1/2at^2: %.2f = %.2f*%.2f + 1/2*%.2f*%.2f^2\n", S, u, t, a, t);

    // equation (iii)
    a = 2.0;
```

```

b = 3.0;
c = 1.0;
T = (2 * a) + sqrt(b) + (9 * c);
printf("T = 2a + sqrt(b) + 9c: %.2f = 2*%.2f + sqrt(%.2f) + 9*%.2f\n", T, a, b, c);

// equation (iv)
b = 4.0;
p = 5.0;
H = sqrt(b * b + p * p);
printf("H = sqrt(b^2 + p^2): %.2f = sqrt(%.2f^2 + %.2f^2)\n", H, b, p);

return 0;
}

```

Output:

```

V = u + at: 16.00 = 10.00 + 2.00*3.00
S = ut + 1/2at^2: 22.00 = 5.00*4.00 + 1/2*1.00*4.00^2
T = 2a + sqrt(b) + 9c: 14.73 = 2*2.00 + sqrt(3.00) + 9*1.00
H = sqrt(b^2 + p^2): 6.40 = sqrt(4.00^2 + 5.00^2)

```

Figure 6 - Equation Outputs

Experiment No. 7

Aim: Write a program to swap two variables:

- a) By using temporary variable.
- b) Without using temporary variable

Concept Used: The concept of swapping variables involves exchanging the values of two variables. This can be achieved using a temporary variable or arithmetic operations. The implementation uses C programming language to demonstrate these methods. Variable swapping is a fundamental concept in programming.

Code:

```
#include <stdio.h>

int main() {

    int x = 10, y = 20, temp;

    printf("Before swapping with temp: x = %d, y = %d\n", x, y);

    temp = x;

    x = y;

    y = temp;

    printf("After swapping with temp: x = %d, y = %d\n", x, y);

    x = 10;

    y = 20;

    printf("Before swapping without temp: x = %d, y = %d\n", x, y);

    x = x + y;

    y = x - y;

    x = x - y;

    printf("After swapping without temp: x = %d, y = %d\n", x, y);
```

```
    return 0;  
}
```

Output:

```
Before swapping with temp: x = 10, y = 20  
After swapping with temp: x = 20, y = 10  
Before swapping without temp: x = 10, y = 20  
After swapping without temp: x = 20, y = 10
```

Figure 7 - Swapping Variables

Experiment No. 8

Aim: Write a Program to find the greatest among three numbers using:

- a) Conditional Operator
- b) If-Else statement

Concept Used: The program utilizes conditional operator and if-else statement in C to determine the greatest among three numbers. This implementation demonstrates basic control structures in C. The conditional operator is a ternary operator used for decision making.

Code:

```
#include <stdio.h>
int main() {
    int a, b, c, greatest;
    printf("Enter first number: ");
    scanf("%d", &a);
    printf("Enter second number: ");
    scanf("%d", &b);
    printf("Enter third number: ");
    scanf("%d", &c);

    greatest = (a > b) ? (a > c ? a : c) : (b > c ? b : c);
    printf("Greatest number using conditional operator: %d\n", greatest);

    if (a >= b && a >= c) {
        greatest = a;
    } else if (b >= a && b >= c) {
        greatest = b;
    } else {
        greatest = c;
    }
    printf("Greatest number using if-else statement: %d\n", greatest);
    return 0;
}
```

Output:

```
Enter first number: 10  
Enter second number: 20  
Enter third number: 30  
Greatest number using conditional operator: 30  
Greatest number using if-else statement: 30
```

Figure 8 - Greatest Number Output

Experiment No. 9

Aim: Write the following programs using switch case statement:

- To check that an input alphabet is vowel or consonant
- To check whether a number is positive, negative or zero

Concept Used: The implementation uses C to demonstrate switch case statements. It involves conditional checks for alphabet and number classification. This approach enables efficient categorization. C programming facilitates this process.

Code:

```
#include <stdio.h>
#include <ctype.h>

void check_alphabet(char c) {
    switch(toupper(c)) {
        case 'A':
        case 'E':
        case 'I':
        case 'O':
        case 'U':
            printf("%c is a vowel\n", c);
            break;
        default:
            if(isalpha(c)) {
                printf("%c is a consonant\n", c);
            } else {
                printf("%c is not an alphabet\n", c);
            }
    }
}

void check_number(int num) {
    switch(num > 0) {
        case 1:
            printf("%d is a positive number\n", num);
    }
}
```

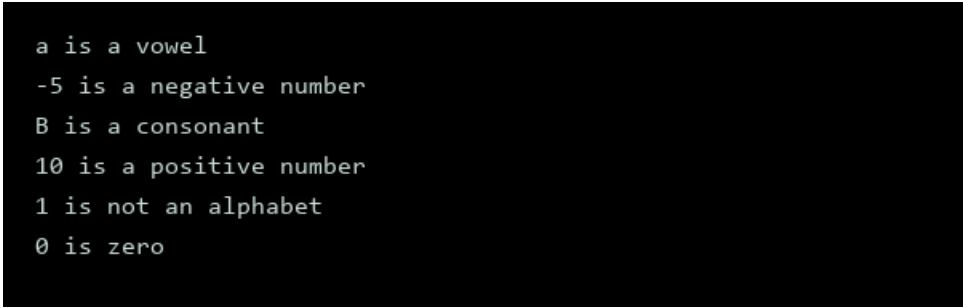
```

        break;
    case 0:
        switch(num) {
            case 0:
                printf("%d is zero\n", num);
                break;
            default:
                printf("%d is a negative number\n", num);
        }
    }
}

int main() {
    char c = 'a';
    int num = -5;
    check_alphabet(c);
    check_number(num);
    c = 'B';
    num = 10;
    check_alphabet(c);
    check_number(num);
    c = '1';
    num = 0;
    check_alphabet(c);
    check_number(num);
    return 0;
}

```

Output:



```

a is a vowel
-5 is a negative number
B is a consonant
10 is a positive number
1 is not an alphabet
0 is zero

```

Figure 9 - Code Execution Output

Experiment No. 10

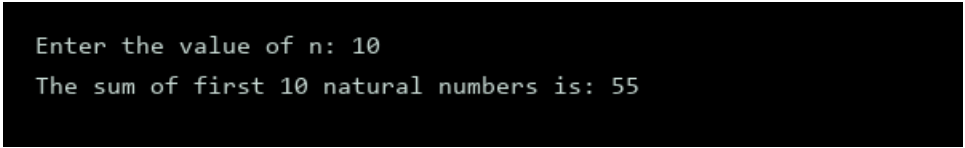
Aim: Write a program using while loop to print the sum of first n natural numbers.

Concept Used: The program utilizes a while loop to calculate the sum of natural numbers. This implementation uses C, demonstrating basic loop control and arithmetic operations. The core concept targeted is iteration and accumulation.

Code:

```
#include <stdio.h>
int main() {
    int n, sum = 0, i = 1;
    printf("Enter the value of n: ");
    scanf("%d", &n);
    while(i <= n) {
        sum += i;
        i++;
    }
    printf("The sum of first %d natural numbers is: %d\n", n, sum);
    return 0;
}
```

Output:

A screenshot of a terminal window with a black background and white text. It shows the program's execution: the prompt 'Enter the value of n: 10' followed by the output 'The sum of first 10 natural numbers is: 55'.

```
Enter the value of n: 10
The sum of first 10 natural numbers is: 55
```

Figure 10 - Natural Number Sum

Experiment No. 11

Aim: Write a program to check a number is Armstrong or not using For loop.

Concept Used: The implementation uses C to check if a number is an Armstrong number, which is a number equal to the sum of its own digits each raised to the power of the number of digits. This concept is achieved through the use of a for loop. The C programming language is utilized for this task.

Code:

```
#include <stdio.h>
#include <math.h>

int main() {
    int num, original, remainder, n = 0, result = 0;
    printf("Enter a number: ");
    scanf("%d", &num);
    original = num;
    while (original != 0) {
        original /= 10;
        n++;
    }
    original = num;
    for (; original != 0; original /= 10) {
        remainder = original % 10;
        result += pow(remainder, n);
    }
    if (result == num)
        printf("%d is an Armstrong number", num);
    else
        printf("%d is not an Armstrong number", num);
    return 0;
}
```

Output:


```
Enter a number: 153  
153 is an Armstrong number
```

Figure 11 - Armstrong Number

Experiment No. 12

Aim: Write the program to count the digits in a number and then print the reverse of the number also.

Concept Used: The implementation uses C to target the concepts of integer manipulation, looping, and conditional statements. It involves counting the digits in a number and reversing the number. This is achieved through arithmetic operations and iterative processes in C.

Code:

```
#include <stdio.h>
int main() {
    int num, reverse = 0, count = 0, remainder;
    printf("Enter a number: ");
    scanf("%d", &num);
    int temp = num;
    while(temp != 0) {
        temp = temp / 10;
        count++;
    }
    printf("Number of digits: %d\n", count);
    temp = num;
    while(temp != 0) {
        remainder = temp % 10;
        reverse = reverse * 10 + remainder;
        temp = temp / 10;
    }
    printf("Reverse of the number: %d\n", reverse);
    return 0;
}
```

Output:

```
Enter a number: 12345  
Number of digits: 5  
Reverse of the number: 54321
```

Figure 12 - Number Reversal Output

Experiment No. 13

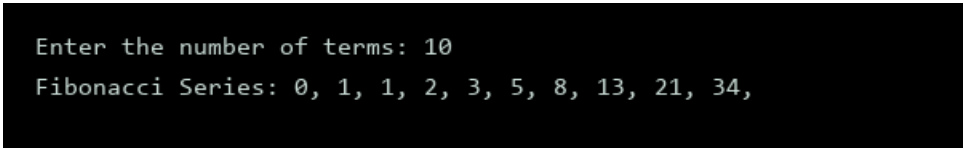
Aim: Write a program to generate the Fibonacci series.

Concept Used: The Fibonacci series is a mathematical concept where each number is the sum of the two preceding ones. This implementation uses C to generate the series. It utilizes loops and arrays to store and print the sequence.

Code:

```
#include <stdio.h>
int main() {
    int n, i;
    printf("Enter the number of terms: ");
    scanf("%d", &n);
    long long fib[n];
    fib[0] = 0;
    fib[1] = 1;
    printf("Fibonacci Series: %lld, %lld, ", fib[0], fib[1]);
    for(i = 2; i < n; i++) {
        fib[i] = fib[i-1] + fib[i-2];
        printf("%lld, ", fib[i]);
    }
    return 0;
}
```

Output:

A screenshot of a terminal window with a black background and white text. The first line shows the prompt 'Enter the number of terms: 10'. The second line shows the output 'Fibonacci Series: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34,'.

```
Enter the number of terms: 10
Fibonacci Series: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34,
```

Figure 13 - Fibonacci Output

Experiment No. 14

Aim: Write a program to print the following patterns:

a)

```
*
* *
* * *
* * * *
* * * * *
* * * * * *
```

b)

```
      *
     * *
    * * *
   * * * *
  * * * * *
 * * * * * *
* * * * * * *
```

Concept Used: The implementation uses C to demonstrate nested loops for pattern generation. Core concepts include loop control and string manipulation. The program utilizes C's printf function for output.

Code:

```
#include <stdio.h>
int main() {
    int i, j;
    // pattern a
    for(i = 1; i <= 6; i++) {
        for(j = 1; j <= i; j++) {
            printf("* ");
        }
        printf("\n");
    }
    printf("\n");
    // pattern b
```

```

for(i = 1; i <= 6; i++) {
    for(j = 1; j <= 6 - i; j++) {
        printf(" ");
    }
    for(j = 1; j <= i; j++) {
        printf("* ");
    }
    printf("\n");
}
return 0;
}

```

Output:

```

*
* *
* * *
* * * *
* * * * *
* * * * *

                                *
                              * *
                             * * *
                            * * * *
                           * * * * *
                          * * * * *
                         * * * * *
                        * * * * *
                       * * * * *
                      * * * * *
                     * * * * *
                    * * * * *
                   * * * * *
                  * * * * *
                 * * * * *
                * * * * *
               * * * * *
              * * * * *
             * * * * *
            * * * * *
           * * * * *
          * * * * *
         * * * * *
        * * * * *
       * * * * *
      * * * * *
     * * * * *
    * * * * *
   * * * * *
  * * * * *
 * * * * *

```

Figure 14 - Pattern Output

Experiment No. 15

Aim: Write the program to print the following pattern:

```
1 2 3 4 5 6
2 4 6 8 10 12
3 6 9 12 15 18
4 8 12 16 20 24
5 10 15 20 25 30
6 12 18 24 30 36
```

Concept Used: The implementation uses C to generate a pattern of multiples. It utilizes nested loops for iteration and multiplication operations to produce the desired output. The core concept targeted is pattern generation through iterative multiplication. C programming language is used for the implementation.

Code:

```
#include <stdio.h>
int main() {
    int i, j;
    for(i = 1; i <= 6; i++) {
        for(j = 1; j <= 6; j++) {
            printf("%d ", i * j);
        }
        printf("\n");
    }
    return 0;
}
```

Output:

```
1 2 3 4 5 6
2 4 6 8 10 12
3 6 9 12 15 18
4 8 12 16 20 24
5 10 15 20 25 30
6 12 18 24 30 36
```

Figure 15 - Pattern Generation Output

Experiment No. 16

Aim: Write a program to check that the given number is prime, Armstrong or perfect using the concept of functions.

Concept Used: The implementation uses C to check if a number is prime, Armstrong, or perfect. It utilizes functions for each type of number check. These concepts are fundamental in number theory and programming.

Code:

```
#include <stdio.h>
#include <math.h>

int is_prime(int num) {
    if (num <= 1) return 0;
    for (int i = 2; i <= sqrt(num); i++) {
        if (num % i == 0) return 0;
    }
    return 1;
}

int is_armstrong(int num) {
    int sum = 0, temp = num;
    while (temp != 0) {
        int digit = temp % 10;
        sum += digit * digit * digit;
        temp /= 10;
    }
    return sum == num;
}

int is_perfect(int num) {
    int sum = 0;
    for (int i = 1; i < num; i++) {
        if (num % i == 0) sum += i;
    }
}
```

```

    return sum == num;
}

int main() {
    int num;
    printf("Enter a number: ");
    scanf("%d", &num);
    printf("%d is ", num);
    if (is_prime(num)) printf("a prime number\n");
    else printf("not a prime number\n");
    printf("%d is ", num);
    if (is_armstrong(num)) printf("an Armstrong number\n");
    else printf("not an Armstrong number\n");
    printf("%d is ", num);
    if (is_perfect(num)) printf("a perfect number\n");
    else printf("not a perfect number\n");
    return 0;
}

```

Output:

```

Enter a number: 371
371 is not a prime number
371 is an Armstrong number
371 is not a perfect number

```

Figure 16 - Number Classification

Experiment No. 17

Aim: Write a program to calculate the area and circumference of a circle using functions.

Concept Used: This program uses functions in C to calculate the area and circumference of a circle. The implementation utilizes the mathematical formulas for area and circumference. C programming language is used for this implementation. Mathematical constants are also used.

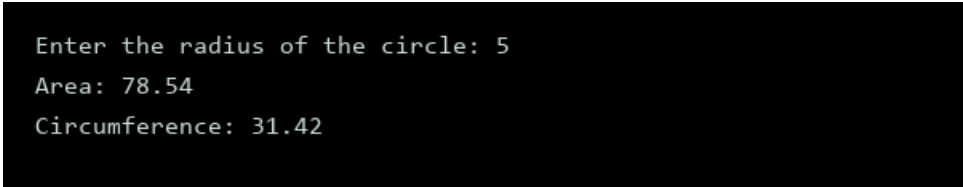
Code:

```
#include <stdio.h>
#include <math.h>

void calculate_circle_properties(double radius, double *area, double *circumference) {
    *area = M_PI * pow(radius, 2);
    *circumference = 2 * M_PI * radius;
}

int main() {
    double radius, area, circumference;
    printf("Enter the radius of the circle: ");
    scanf("%lf", &radius);
    calculate_circle_properties(radius, &area, &circumference);
    printf("Area: %.2f\n", area);
    printf("Circumference: %.2f\n", circumference);
    return 0;
}
```

Output:

A screenshot of a terminal window with a black background and light green text. It shows the output of the C program: 'Enter the radius of the circle: 5', 'Area: 78.54', and 'Circumference: 31.42'.

```
Enter the radius of the circle: 5
Area: 78.54
Circumference: 31.42
```

Figure 17 - Circle Properties

Experiment No. 18

Aim: Write a program to swap two variables using the concept of call by value and call by reference.

Concept Used: The concept of call by value and call by reference is used to swap two variables. In C, call by value involves passing a copy of the variable, while call by reference involves passing the memory address. This implementation uses C to demonstrate these concepts.

Code:

```
#include <stdio.h>
void swap_call_by_value(int a, int b) {
    int temp = a;
    a = b;
    b = temp;
    printf("Inside call_by_value: a = %d, b = %d\n", a, b);
}
void swap_call_by_reference(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
    printf("Inside call_by_reference: a = %d, b = %d\n", *a, *b);
}
int main() {
    int x = 10;
    int y = 20;
    printf("Before swapping: x = %d, y = %d\n", x, y);
    swap_call_by_value(x, y);
    printf("After call_by_value: x = %d, y = %d\n", x, y);
    swap_call_by_reference(&x, &y);
    printf("After call_by_reference: x = %d, y = %d\n", x, y);
    return 0;
}
```

Output:

```
Before swapping: x = 10, y = 20  
Inside call_by_value: a = 20, b = 10  
After call_by_value: x = 10, y = 20  
Inside call_by_reference: a = 20, b = 10  
After call_by_reference: x = 20, y = 10
```

Figure 18 - Swap Output

Experiment No. 19

Aim: Write a program to perform the following operations on 1D-Array:

- Insert
- Update
- Delete
- Display
- Search

Concept Used: The implementation uses C to target fundamental array operations. Core concepts include dynamic memory allocation and array manipulation. These operations are essential in programming. The implementation demonstrates insert, update, delete, display, and search functions.

Code:

```
#include <stdio.h>
#include <stdlib.h>

void insert(int *arr, int *size, int element) {
    *size = *size + 1;
    arr = realloc(arr, *size * sizeof(int));
    arr[*size - 1] = element;
}

void update(int *arr, int size, int index, int element) {
    if(index < size) {
        arr[index] = element;
    }
}

void delete(int *arr, int *size, int index) {
    if(index < *size) {
        for(int i = index; i < *size - 1; i++) {
            arr[i] = arr[i + 1];
        }
        *size = *size - 1;
    }
}
```

```

        arr = realloc(arr, *size * sizeof(int));
    }
}

void display(int *arr, int size) {
    for(int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

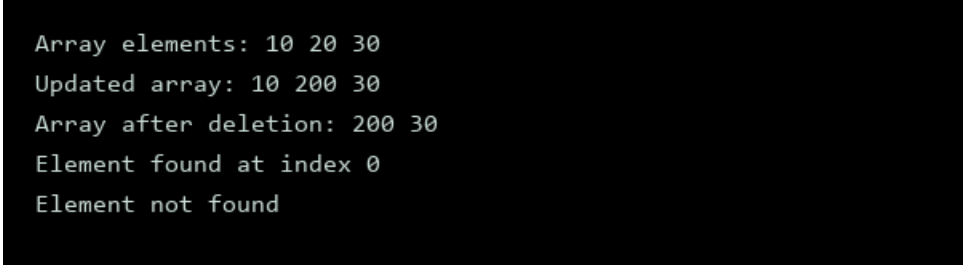
void search(int *arr, int size, int element) {
    int found = 0;
    for(int i = 0; i < size; i++) {
        if(arr[i] == element) {
            printf("Element found at index %d\n", i);
            found = 1;
        }
    }
    if(!found) {
        printf("Element not found\n");
    }
}

int main() {
    int size = 0;
    int *arr = NULL;
    insert(&arr, &size, 10);
    insert(&arr, &size, 20);
    insert(&arr, &size, 30);
    printf("Array elements: ");
    display(arr, size);
    update(arr, size, 1, 200);
    printf("Updated array: ");
    display(arr, size);
    delete(arr, &size, 0);
    printf("Array after deletion: ");
    display(arr, size);
    search(arr, size, 200);
    search(arr, size, 10);
}

```

```
    return 0;  
}
```

Output:



```
Array elements: 10 20 30  
Updated array: 10 200 30  
Array after deletion: 200 30  
Element found at index 0  
Element not found
```

Figure 19 - Array Operations Output

Experiment No. 20

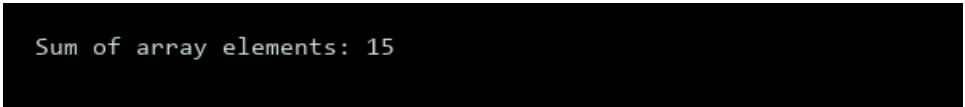
Aim: Write a program to calculate the sum of array elements by passing it to a function.

Concept Used: The problem uses arrays and functions in C, implementing basic algorithmic concepts. It calculates the sum of array elements by passing the array to a function, demonstrating parameter passing in C. The implementation utilizes C programming language.

Code:

```
#include <stdio.h>
int sum_array(int arr[], int size) {
    int sum = 0;
    for (int i = 0; i < size; i++) {
        sum += arr[i];
    }
    return sum;
}
int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int size = sizeof(arr) / sizeof(arr[0]);
    printf("Sum of array elements: %d\n", sum_array(arr, size));
    return 0;
}
```

Output:



```
Sum of array elements: 15
```

Figure 20 - Array Sum Output

Experiment No. 21

Aim: Write a program to show the use of passing pointer as arguments to the functions.

Concept Used: The concept of passing pointers as arguments to functions in C allows for modification of original variables. This is achieved by passing the memory address of a variable to a function. The implementation uses C to demonstrate this concept. Pointers enable efficient data manipulation.

Code:

```
#include <stdio.h>
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
int main() {
    int x = 5;
    int y = 10;
    printf("Before swap: x = %d, y = %d\n", x, y);
    swap(&x, &y);
    printf("After swap: x = %d, y = %d\n", x, y);
    return 0;
}
```

Output:



```
Before swap: x = 5, y = 10
After swap: x = 10, y = 5
```

Figure 21 - Pointer Swap Output

Experiment No. 22

Aim: Write a program matrix multiplication using the concept of 2D array.

Concept Used: Matrix multiplication is a fundamental concept in linear algebra, involving the multiplication of two matrices to produce another matrix. This implementation uses C to demonstrate the concept of 2D array multiplication. The program takes two matrices as input and produces the resulting matrix.

Code:

```
#include <stdio.h>

void multiplyMatrices(int mat1[2][2], int mat2[2][2], int result[2][2]) {
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            result[i][j] = 0;
            for (int k = 0; k < 2; k++) {
                result[i][j] += mat1[i][k] * mat2[k][j];
            }
        }
    }
}

int main() {
    int mat1[2][2] = {{1, 2}, {3, 4}};
    int mat2[2][2] = {{5, 6}, {7, 8}};
    int result[2][2];

    multiplyMatrices(mat1, mat2, result);

    printf("Matrix 1:\n");
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            printf("%d ", mat1[i][j]);
        }
        printf("\n");
    }
}
```

```

    }

    printf("Matrix 2:\n");
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            printf("%d ", mat2[i][j]);
        }
        printf("\n");
    }

    printf("Result:\n");
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            printf("%d ", result[i][j]);
        }
        printf("\n");
    }

    return 0;
}

```

Output:

```

Matrix 1:
1 2
3 4
Matrix 2:
5 6
7 8
Result:
19 22
43 50

```

Figure 22 - Matrix Multiplication

Experiment No. 23

Aim: Write a program to transpose a given matrix.

Concept Used: Matrix transposition involves swapping rows with columns. This implementation uses C to achieve the desired result. It utilizes nested loops for iteration and temporary storage for the transposed matrix.

Code:

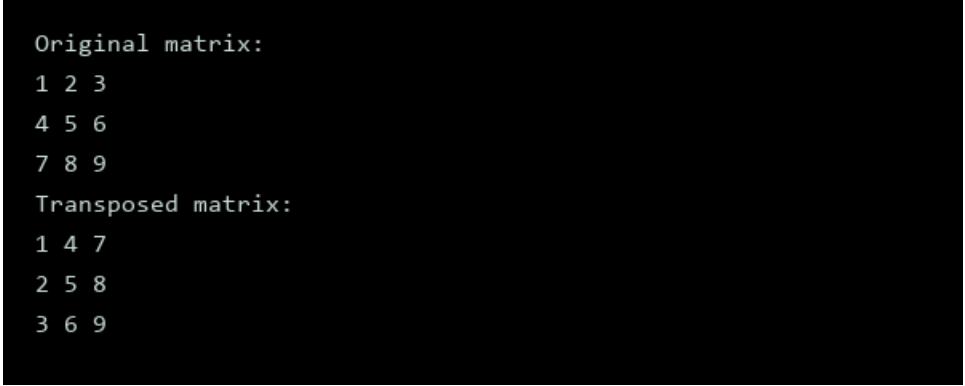
```
#include <stdio.h>
#define ROWS 3
#define COLS 3

void transpose(int matrix[ROWS][COLS]) {
    int transposed[COLS][ROWS];
    for (int i = 0; i < ROWS; i++) {
        for (int j = 0; j < COLS; j++) {
            transposed[j][i] = matrix[i][j];
        }
    }
    printf("Transposed matrix:\n");
    for (int i = 0; i < COLS; i++) {
        for (int j = 0; j < ROWS; j++) {
            printf("%d ", transposed[i][j]);
        }
        printf("\n");
    }
}

int main() {
    int matrix[ROWS][COLS] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
    printf("Original matrix:\n");
    for (int i = 0; i < ROWS; i++) {
        for (int j = 0; j < COLS; j++) {
            printf("%d ", matrix[i][j]);
        }
    }
}
```

```
    printf("\n");  
}  
transpose(matrix);  
return 0;  
}
```

Output:



```
Original matrix:  
1 2 3  
4 5 6  
7 8 9  
Transposed matrix:  
1 4 7  
2 5 8  
3 6 9
```

Figure 23 - Matrix Transposed

Experiment No. 24

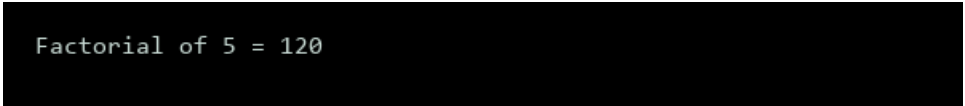
Aim: Write a program to find the factorial of a number by using the concept of recursion.

Concept Used: The concept of recursion is used to find the factorial of a number. Recursion involves a function calling itself until a base condition is met. This implementation uses C to demonstrate the recursive approach.

Code:

```
#include <stdio.h>
int factorial(int n) {
    if (n == 0 || n == 1)
        return 1;
    else
        return n * factorial(n - 1);
}
int main() {
    int num = 5;
    printf("Factorial of %d = %d\n", num, factorial(num));
    return 0;
}
```

Output:



```
Factorial of 5 = 120
```

Figure 24 - Factorial Result

Experiment No. 25

Aim: Write a menu driven C program to show the use of in-built string functions like strlen, strcat, strcpy, strcmp, strrev etc.

Concept Used: The implementation uses C to demonstrate string functions. It utilizes built-in functions such as strlen, strcat, strcpy, strcmp, and strrev. These functions enable various string operations like length calculation, concatenation, copying, comparison, and reversal. C programming language is used for the implementation.

Code:

```
#include <stdio.h>
#include <string.h>

void displayMenu() {
    printf("1. String Length\n");
    printf("2. String Concatenation\n");
    printf("3. String Copy\n");
    printf("4. String Comparison\n");
    printf("5. String Reversal\n");
    printf("6. Exit\n");
}

int main() {
    char str1[100], str2[100], str3[100];
    int choice, len;
    while (1) {
        displayMenu();
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Enter a string: ");
                scanf("%s", str1);
                len = strlen(str1);
                printf("Length of the string: %d\n", len);
```



```

        break;
case 2:
    printf("Enter first string: ");
    scanf("%s", str1);
    printf("Enter second string: ");
    scanf("%s", str2);
    strcat(str1, str2);
    printf("Concatenated string: %s\n", str1);
    break;
case 3:
    printf("Enter a string: ");
    scanf("%s", str1);
    strcpy(str2, str1);
    printf("Copied string: %s\n", str2);
    break;
case 4:
    printf("Enter first string: ");
    scanf("%s", str1);
    printf("Enter second string: ");
    scanf("%s", str2);
    len = strcmp(str1, str2);
    if (len == 0)
        printf("Strings are equal\n");
    else if (len < 0)
        printf("First string is lexicographically smaller\n");
    else
        printf("Second string is lexicographically smaller\n");
    break;
case 5:
    printf("Enter a string: ");
    scanf("%s", str1);
    printf("Reversed string: ");
    for (int i = strlen(str1) - 1; i >= 0; i--)
        printf("%c", str1[i]);
    printf("\n");
    break;
case 6:
    return 0;
default:
    printf("Invalid choice\n");

```

```
    }  
}  
return 0;  
}
```

Output:

```
1. String Length  
2. String Concatenation  
3. String Copy  
4. String Comparison  
5. String Reversal  
6. Exit  
Enter your choice: 1  
Enter a string: Hello  
Length of the string: 5  
1. String Length  
2. String Concatenation  
3. String Copy  
4. String Comparison  
5. String Reversal  
6. Exit  
Enter your choice: 2  
Enter first string: Hello  
Enter second string: World  
Concatenated string: HelloWorld  
1. String Length  
2. String Concatenation  
3. String Copy  
4. String Comparison  
5. String Reversal  
6. Exit  
Enter your choice: 6
```

Figure 25 - Menu Driven Output

Experiment No. 26

Aim: Write a Program in C to display the total number of appearances of a substring provided as input by the user in a given string.

Concept Used: The program utilizes string manipulation and searching techniques in C. It employs a loop to iterate through the string, checking for occurrences of the substring. This implementation uses C, leveraging its standard library functions for input and output operations. The program demonstrates basic string handling.

Code:

```
#include <stdio.h>
#include <string.h>

int count_substring(const char *str, const char *sub) {
    int count = 0;
    int len = strlen(sub);
    const char *p = str;

    while ((p = strstr(p, sub)) != NULL) {
        count++;
        p += len;
    }

    return count;
}

int main() {
    char str[100], sub[100];

    printf("Enter the string: ");
    fgets(str, sizeof(str), stdin);
    str[strcspn(str, "\n")] = 0;

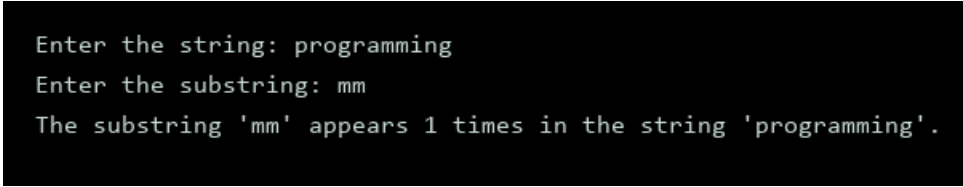
    printf("Enter the substring: ");
    fgets(sub, sizeof(sub), stdin);
```

```
sub[strcspn(sub, "\n")] = 0;

int count = count_substring(str, sub);
printf("The substring '%s' appears %d times in the string '%s'.\n", sub, count, str);

return 0;
}
```

Output:



```
Enter the string: programming
Enter the substring: mm
The substring 'mm' appears 1 times in the string 'programming'.
```

Figure 26 - Substring Count Output

Experiment No. 27

Aim: Write a program to display the sum of the digits of a number by using the concept of recursion.

Concept Used: Recursion is utilized to break down a number into individual digits and sum them. This implementation uses C, demonstrating the concept of recursive functions. The program takes an integer as input and calculates the sum of its digits using recursive calls.

Code:

```
#include <stdio.h>
int sum_of_digits(int n) {
    if(n == 0)
        return 0;
    return (n % 10) + sum_of_digits(n / 10);
}
int main() {
    int num;
    printf("Enter a number: ");
    scanf("%d", &num);
    printf("Sum of digits: %d\n", sum_of_digits(num));
    return 0;
}
```

Output:

A screenshot of a terminal window with a black background and white text. It shows the program's output for the input 12345.

```
Enter a number: 12345
Sum of digits: 15
```

Figure 27 - Sum of Digits

Experiment No. 28

Aim: Write a C program to add two distances in inch & feet using the concept of structures.

Concept Used: This program utilizes structures in C to represent distances in feet and inches, demonstrating object-oriented programming principles. The implementation uses C to define and manipulate these structured data types. The concept of structures allows for organized data storage and efficient operations.

Code:

```
#include <stdio.h>

typedef struct {
    int feet;
    int inch;
} Distance;

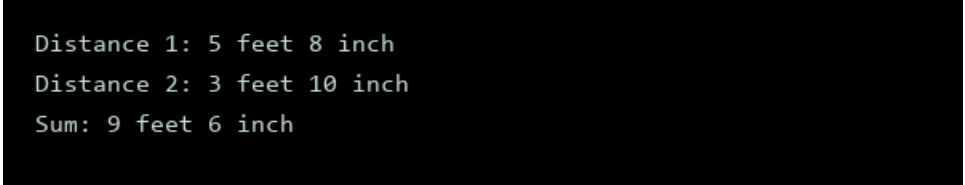
Distance addDistance(Distance d1, Distance d2) {
    Distance sum;
    sum.inch = d1.inch + d2.inch;
    sum.feet = d1.feet + d2.feet + sum.inch / 12;
    sum.inch %= 12;
    return sum;
}

void printDistance(Distance d) {
    printf("%d feet %d inch\n", d.feet, d.inch);
}

int main() {
    Distance d1 = {5, 8};
    Distance d2 = {3, 10};
    Distance sum = addDistance(d1, d2);
    printf("Distance 1: ");
    printDistance(d1);
```

```
printf("Distance 2: ");  
printDistance(d2);  
printf("Sum: ");  
printDistance(sum);  
return 0;  
}
```

Output:



```
Distance 1: 5 feet 8 inch  
Distance 2: 3 feet 10 inch  
Sum: 9 feet 6 inch
```

Figure 28 - Distance Addition Output

Experiment No. 29

Aim: Write a C program to add two complex numbers using the concept of structures in C.

Concept Used: The implementation uses C to create a structure for complex numbers, allowing for the addition of two complex numbers. This concept targets the use of structures in C for data encapsulation. The program will define a structure with real and imaginary components.

Code:

```
#include <stdio.h>
struct complex {
    float real;
    float imag;
};
struct complex addComplex(struct complex c1, struct complex c2) {
    struct complex temp;
    temp.real = c1.real + c2.real;
    temp.imag = c1.imag + c2.imag;
    return temp;
}
int main() {
    struct complex c1, c2, sum;
    printf("Enter real part of first complex number: ");
    scanf("%f", &c1.real);
    printf("Enter imaginary part of first complex number: ");
    scanf("%f", &c1.imag);
    printf("Enter real part of second complex number: ");
    scanf("%f", &c2.real);
    printf("Enter imaginary part of second complex number: ");
    scanf("%f", &c2.imag);
    sum = addComplex(c1, c2);
    printf("Sum of two complex numbers = %.2f + %.2fi\n", sum.real, sum.imag);
    return 0;
}
```


Output:

```
Enter real part of first complex number: 5
Enter imaginary part of first complex number: 7
Enter real part of second complex number: 3
Enter imaginary part of second complex number: 2
Sum of two complex numbers = 8.00 + 9.00i
```

Figure 29 - Complex Number Sum

Experiment No. 30

Aim: Write a program in C to store the information of five employees using both concepts i.e. array of structure and array within structure.

Concept Used: The implementation uses C to demonstrate array of structure and array within structure concepts. This program stores employee information utilizing these concepts. The code showcases C programming language capabilities.

Code:

```
#include <stdio.h>
#include <string.h>

struct Employee {
    int id;
    char name[100];
    char department[100];
};

struct Employee2 {
    int id;
    char name[100];
    char departments[5][100];
};

int main() {
    struct Employee employees[5];
    struct Employee2 employee2[5];

    // Storing information using array of structure
    for (int i = 0; i < 5; i++) {
        printf("Enter id for employee %d: ", i+1);
        scanf("%d", &employees[i].id);
        printf("Enter name for employee %d: ", i+1);
        scanf("%s", employees[i].name);
        printf("Enter department for employee %d: ", i+1);
```

```

        scanf("%s", employees[i].department);
    }

    printf("\nInformation using array of structure:\n");
    for (int i = 0; i < 5; i++) {
        printf("Employee %d id: %d\n", i+1, employees[i].id);
        printf("Employee %d name: %s\n", i+1, employees[i].name);
        printf("Employee %d department: %s\n", i+1, employees[i].department);
    }

    // Storing information using array within structure
    for (int i = 0; i < 5; i++) {
        printf("Enter id for employee %d: ", i+1);
        scanf("%d", &employee2[i].id);
        printf("Enter name for employee %d: ", i+1);
        scanf("%s", employee2[i].name);
        for (int j = 0; j < 5; j++) {
            printf("Enter department %d for employee %d: ", j+1, i+1);
            scanf("%s", employee2[i].departments[j]);
        }
    }

    printf("\nInformation using array within structure:\n");
    for (int i = 0; i < 5; i++) {
        printf("Employee %d id: %d\n", i+1, employee2[i].id);
        printf("Employee %d name: %s\n", i+1, employee2[i].name);
        for (int j = 0; j < 5; j++) {
            printf("Employee %d department %d: %s\n", i+1, j+1,
employee2[i].departments[j]);
        }
    }

    return 0;
}

```

Output:

```
Enter id for employee 1: 101
Enter name for employee 1: John
Enter department for employee 1: Sales
Enter id for employee 2: 102
Enter name for employee 2: Alice
Enter department for employee 2: Marketing
Enter id for employee 3: 103
Enter name for employee 3: Bob
Enter department for employee 3: IT
Enter id for employee 4: 104
Enter name for employee 4: Charlie
Enter department for employee 4: Finance
Enter id for employee 5: 105
Enter name for employee 5: David
Enter department for employee 5: HR

Information using array of structure:
Employee 1 id: 101
Employee 1 name: John
Employee 1 department: Sales
Employee 2 id: 102
Employee 2 name: Alice
Employee 2 department: Marketing
Employee 3 id: 103
Employee 3 name: Bob
Employee 3 department: IT
Employee 4 id: 104
Employee 4 name: Charlie
Employee 4 department: Finance
Employee 5 id: 105
Employee 5 name: David
Employee 5 department: HR
```

Figure 30 - Employee Information

Experiment No. 31

Aim: Write a Program in C to find and replace a specific string in a file and also display the total number of appearances of that string.

Concept Used: The implementation uses C to find and replace a specific string in a file. It utilizes file input/output operations and string manipulation functions. The program reads a file, replaces the target string, and displays the total number of appearances. C programming language is used for this task.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int count_and_replace(const char *filename, const char *old_str, const char *new_str) {
    FILE *file = fopen(filename, "r");
    if (!file) return -1;

    char line[1024];
    int count = 0;
    char temp[] = "temp.txt";

    FILE *temp_file = fopen(temp, "w");
    if (!temp_file) {
        fclose(file);
        return -1;
    }

    while (fgets(line, sizeof(line), file)) {
        char *ptr = strstr(line, old_str);
        while (ptr) {
            count++;
            char *rest = strstr(ptr + 1, old_str);
            if (rest) {
                strncpy(ptr, new_str, rest - ptr);
            }
            ptr = rest;
        }
        fprintf(temp_file, "%s\n", line);
    }
    fclose(file);
    fclose(temp_file);
    return count;
}
```

```

        ptr = rest;
    } else {
        strncpy(ptr, new_str, strlen(old_str));
        break;
    }
}
fputs(line, temp_file);
}

fclose(file);
fclose(temp_file);

remove(filename);
rename(temp, filename);

return count;
}

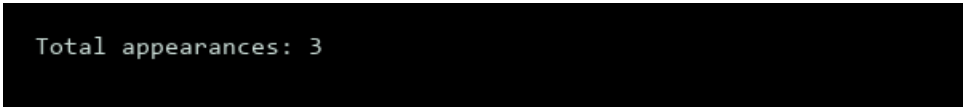
int main() {
    const char *filename = "example.txt";
    const char *old_str = "old";
    const char *new_str = "new";

    int count = count_and_replace(filename, old_str, new_str);
    if (count != -1) {
        printf("Total appearances: %d\n", count);
    } else {
        printf("Error occurred\n");
    }

    return 0;
}

```

Output:



```
Total appearances: 3
```

Figure 31 – Total Appearances Result